Cascading Discreet Log Contracts

Pedro Martins Rodrigues Novaes, GPT 5.5

Abstract

Cascading Discreet Log Contracts, or cDLCs, are a construction for composing DLCs into finite graphs of conditionally activated Bitcoin transactions. For each parent outcome x, the oracle attestation point S_x is used as the adaptor point for signatures on a bridge transaction that spends the parent outcome output into a prepared child funding output. When the oracle later reveals the attestation scalar s_x, the same scalar that settles the parent outcome also completes the bridge signatures and funds the child DLC.

Contents

1 Notation **2**

2 Signature Adaptors **2**

3 A DLC as a Conditional Signature Source **2**

4 cDLC Construction **3**

5 Native Chaining **3**

6 Security Claims **3**

7 Machine-Checked Algebra **4**

8 Refunds and Failure Modes **5**
- 8.1 Assumptions . . . 5
- 8.2 Timeout Ordering . . . 5

9 Lazy Graph Preparation **5**

10 Financial Completeness With Lazy Compression **6**

11 Applications **6**

12 Lightning Network Extension **7**
- 12.1 Channel Condition Model . . . 7
- 12.2 HTLC Encoding . . . 7
- 12.3 Routed HTLC cDLCs . . . 7
- 12.4 Point-Locked Encoding . . . 7
- 12.5 Routed Point-Locked cDLCs . . . 8
- 12.6 Machine-Checked Lightning Model . . . 8

13 Limitations **8**
- 13.1 Cryptographic Assumptions . . . 8
- 13.2 Oracle and Liveness Risk . . . 8
- 13.3 State and Negotiation Cost . . . 8
- 13.4 Bitcoin Policy, Fees, and Timelocks . . . 8
- 13.5 Privacy Limits . . . 8
- 13.6 Collateral Requirements . . . 9
- 13.7 Scope of the Formal Claim . . . 9

14 Conclusion **9**

1. Notation

Let G be the generator of the secp256k1 group of order n. Lowercase letters denote scalars modulo n; uppercase letters denote curve points. For a scalar x, write $X = xG$. Let H denote a tagged hash interpreted as a scalar modulo n.

A BIP340 Schnorr signature for message m under public key $P = pG$ is a pair (R, s) satisfying

$$sG = R + H(R \,\|\, P \,\|\, m)\, P.$$

An oracle has signing key v, public key $V = vG$, and commits to a one-time nonce point $R_o = r_o G$ for an event. For outcome x, define

$$\begin{aligned} e_x &= H(R_o \,\|\, V \,\|\, x), \\ s_x &= r_o + e_x v \pmod{n}, \\ S_x &= s_x G = R_o + e_x V. \end{aligned}$$

Before the event, anyone can compute S_x for each outcome x, but only the oracle knows s_x. After the event, the oracle attests by publishing s_x. This scalar is the transferable fact that makes DLCs possible.

2. Signature Adaptors

Let $T = tG$ be a public point whose discrete logarithm t is unknown. To create an adaptor signature for message m under key $P = pG$, the signer chooses nonce r and computes

$$\begin{aligned} R &= rG, \quad R^* = R + T, \\ e &= H(R^* \,\|\, P \,\|\, m), \\ \hat{s} &= r + ep \pmod{n}. \end{aligned}$$

The adaptor $(R^*, T, \hat{s})$ can be verified without knowing t:

$$\hat{s}\, G = R^* - T + eP.$$

It is not yet a valid Schnorr signature. Once t is revealed, anyone holding the adaptor computes $s = \hat{s} + t \pmod{n}$ and (R^*, s) is valid because

$$sG = (\hat{s} + t)G = (R^* - T + eP) + T = R^* + eP.$$

Conversely, a completed signature reveals the hidden scalar to any holder of the adaptor:

$$t = s - \hat{s} \pmod{n}.$$

3. A DLC as a Conditional Signature Source

In a regular DLC, Alice and Bob lock funds into a funding output and prepare a set of Contract Execution Transactions (CETs). Each CET corresponds to an outcome x and pays according to a payout function $\pi(x)$.

For each outcome x, the parties use the oracle attestation point S_x as the adaptor point for the CET signatures. The CET for x cannot be completed until the oracle publishes s_x.

Thus a DLC does more than choose a payout: it emits a scalar

$$\text{resolve}(\text{DLC}, x) \longrightarrow s_x.$$

This scalar is public after resolution, unpredictable before, and bound to a specific oracle event and outcome. A cDLC uses this scalar as the activation secret for the next transaction.

4. cDLC Construction

A cDLC is a finite directed graph $\Gamma = (N, E)$ where each node $C_i \in N$ is a DLC and each directed edge

$$e = (C_i,\, x,\, C_j)$$

means: if contract C_i resolves to outcome x, activate contract C_j.

For each edge $e = (C_i, x, C_j)$ the parties construct a *bridge transaction* B_e. This transaction spends a designated output of the parent CET $\mathrm{CET}_{i,x}$ and creates the funding output F_j of the child DLC C_j. The parent CET contains an *edge output* O_e spendable by the required cDLC participants, with a timelocked refund path.

The child contract C_j is negotiated before the parent resolves. Its CETs and refund transaction spend F_j, which is possible because the `txid` of B_e is known before its witness signatures are completed.

All signatures on B_e are adaptor signatures using $T_e = S_{i,x}$. For participant a with key $P_a = p_a G$, let m_e be the sighash for B_e. The participant computes

$$\begin{aligned} R_a &= r_a G, \quad R_a^* = R_a + S_{i,x}, \\ e_a &= H(R_a^* \,\|\, P_a \,\|\, m_e), \\ \hat{s}_a &= r_a + e_a p_a \pmod{n}. \end{aligned}$$

The adaptor verifies as $\hat{s}_a G = R_a^* - S_{i,x} + e_a P_a$. Once the oracle publishes $s_{i,x}$, anyone holding the adaptors completes:

$$s_a = \hat{s}_a + s_{i,x} \pmod{n},$$

yielding $s_a G = R_a^* + e_a P_a$, a valid Schnorr signature.

Every adaptor on B_e must be exchanged and verified before $\mathrm{Funding}_i$ is broadcast. The completability step requires at least one honest party to retain the adaptor set; the oracle scalar alone is insufficient if all adaptor holders are unavailable.

5. Native Chaining

The bridge transaction spends an output of a *future* CET. This is possible because the `txid` of a SegWit or Taproot transaction does not include witness data: the parent CET can be fully specified before its witness signatures are completed.

Thus the bridge and the child DLC are constructed before the parent CET is broadcast:

$$\mathrm{Funding}_i \;\to\; \mathrm{CET}_{i,x} \;\to\; B_e \;\to\; \mathrm{Funding}_j.$$

The off-chain cascade reads:

$$\begin{aligned} &\text{oracle signs } x \;\Rightarrow\; s_{i,x} \text{ revealed} \;\Rightarrow\; \mathrm{CET}_{i,x} \text{ executable} \\ &\Rightarrow\; B_e \text{ completable} \;\Rightarrow\; C_j \text{ funded.} \end{aligned}$$

Bitcoin validates only ordinary signatures and timelocks. The chain of meaning is entirely off-chain.

6. Security Claims

Claim 6.1 (Conditional Activation)**.** *For an edge $e = (C_i, x, C_j)$, the bridge B_e cannot be completed before $s_{i,x}$ is known, except by forging a Schnorr signature or solving the discrete logarithm of $S_{i,x}$.*

Proof. Each required signature on B_e is an adaptor with point $S_{i,x}$. Completing it without $s_{i,x}$ requires producing a valid Schnorr signature for m_e under P_a (contradicting unforgeability) or finding $\log_G S_{i,x}$. □

Claim 6.2 (Public Completion After Resolution)**.** *If the oracle publishes $s_{i,x}$, any party holding the bridge adaptors can complete B_e.*

Proof. For every required signature: $s_a = \hat{s}_a + s_{i,x}$. Since $S_{i,x} = s_{i,x}G$, we have $s_aG = (\hat{s}_a + s_{i,x})G = R_a^* + e_aP_a$, a valid Schnorr signature. □

Claim 6.3 (Outcome Isolation)**.** *If the oracle signs outcome $y \neq x$ under the same nonce R_o, the scalar $s_{i,y}$ does not complete the bridge for edge (C_i, x, C_j).*

Proof. Completing with $s_{i,y}$ yields $(\hat{s}_a + s_{i,y})G = R_a^* - S_{i,x} + e_aP_a + S_{i,y}$, which equals $R_a^* + e_aP_a$ only if $S_{i,y} = S_{i,x}$. With unique oracle nonces and collision-resistant tagged hashes, this occurs only with negligible probability. □

Remark 6.1 (Oracle Equivocation)**.** *Claim 3 is conditional on the oracle using a single nonce per event.*

Same-nonce equivocation*: the oracle signs outcomes x and y with the same R_o. Only one bridge completes; additionally, the oracle's private key v is recoverable.*

Different-nonce equivocation*: the oracle signs two outcomes using independent nonce pairs. Claim 3's isolation argument does not apply; both child edges could become funded simultaneously. In both cases the conflicting signatures are publicly verifiable evidence of oracle misconduct.*

7. Machine-Checked Algebra

The algebra above is modelled in Ada/SPARK and checked with GNATprove. Proof artifacts are in the repository's `spark/` directory.

Targets.

- `spark/cdlc_integer_proofs.gpr` — symbolic polynomial identities over `SPARK.Big_Integers`.
- `spark/cdlc_residue_proofs.gpr` — bridge/adaptor identities over $\mathbb{Z}/97\mathbb{Z}$ with explicit modular reduction.
- `spark/cdlc_proofs.gpr` — same identities with Ada `type mod 97`, using ghost lemmas for modular rotation and cancellation.
- `spark/lazy_cdlc_financial_completeness_proofs.gpr` — conservative branch-step accounting for computable, collateral-bounded financial transitions under Lazy Compression.

The cDLC algebra models prove:

1. The oracle scalar maps to the advertised attestation point.
2. A bridge adaptor signature verifies before completion.
3. Adding the oracle scalar completes the bridge signature.
4. A completed signature reveals the hidden scalar by subtraction.
5. A different oracle scalar does not complete the same bridge signature.

These correspond exactly to $S_x = s_xG$, $\hat{s}G = R^* - S_x + eP$, $s = \hat{s} + s_x$, $sG = R^* + eP$, $s - \hat{s} = s_x$.

The Lazy financial-completeness model proves the finite branch-step obligations used by Section 10: positive and negative financial flows are conservative redistributions, prepared matching branches activate, wrong or unprepared branches fall back, equivalent compressed

states share a continuation template, in-window prepared branches are executable, and terminal payout accounting conserves funded collateral.

All targets cited above completed with no unproved checks and no `pragma Assume` statements, using CVC5, Z3, and Alt-Ergo. The proofs establish algebraic consistency of these identities and finite accounting predicates; secp256k1, BIP340, SHA-256, transaction serialisation, computability theory, and the full production protocol remain outside the machine-checked boundary.

8. Refunds and Failure Modes

8.1 Assumptions

The construction requires:

1. The oracle uses a unique nonce per event and does not sign conflicting outcomes for the same nonce commitment.
2. The oracle publishes the attestation before the fallback timeout.
3. At least one party retains pre-signed transactions and adaptor signatures for every live edge in the preparation window.
4. Fee rates are handled by CPFP, anchor outputs, or pre-agreed fee reserves.
5. Timelocks are ordered so that parent settlement, bridge activation, and child refunds do not race.

8.2 Timeout Ordering

Let τ_{attest} be the expected oracle attestation time, τ_{bridge} the bridge activation deadline, and τ_{child} the child refund timeout. The required ordering is:

$$\tau_{\text{attest}} < \tau_{\text{bridge}} < \tau_{\text{child}}.$$

9. Lazy Graph Preparation

The cDLC activation equation is *edge-local*: preparing bridge B_e for edge (C_i, x, C_j) depends only on the parent outcome, the bridge transaction, and the child funding output for that edge. This locality allows lazy preparation.

Let $K \geq 2$ be a preparation-window depth. At active node C_i, the live window is $W_i = \{C_i, C_{i+1}, \ldots, C_{i+K-1}\}$. Before C_i resolves, the parties negotiate the next window by exchanging adaptor signatures for C_{i+K} — an asynchronous bilateral exchange identical in structure to any DLC negotiation, requiring no simultaneous presence. If no extension is agreed, the fallback executes.

State bounds. For a non-recombining tree with branching factor b, depth D, and per-node weight P:

$$\text{EagerState}(D) = P\,\frac{b^{D+1}-1}{b-1}, \qquad \text{LazyState}(K) = P\,\frac{b^{K}-1}{b-1}.$$

For fixed K, $\text{LazyState}(K)$ is independent of D: maximum live retained state is bounded regardless of total product depth. If reachable states recombine, let $|\Sigma_h|$ be the distinct states at distance h; then

$$\text{LiveState}_i(K) \;\leq\; \sum_{h=0}^{K-1} |\Sigma_h|\, P_h.$$

Recombination, payoff compression, and lazy preparation are compatible and jointly reduce retained state.

10. Financial Completeness With Lazy Compression

A financial contract over a finite oracle alphabet Σ can be represented as a state machine. Let

$$z = (q, m, a, b, r)$$

where q is finite control, m is encoded contract memory, a is Alice's claim, b is Bob's claim, and r is reserve collateral. The funded collateral is conserved:

$$a + b + r = T.$$

Let

$$\delta : Z \times \Sigma \to Z$$

be a computable next-state function, and let

$$\sigma : Z \to \mathrm{Option}(A_{\mathrm{payout}}, B_{\mathrm{payout}})$$

be a computable terminal-payout function. The contract is collateral-bounded when every terminal payout satisfies

$$A_{\mathrm{payout}} + B_{\mathrm{payout}} \leq T.$$

For each compressed state class $\kappa(z)$ and oracle outcome x, the compiler prepares one cDLC branch

$$B_{\kappa(z),x} : z \to \delta(z, x).$$

The bridge for that branch is adapted to the oracle attestation point S_x. When the oracle publishes s_x, the matching branch becomes completable and funds the child state. A non-matching outcome scalar does not complete that branch by outcome isolation.

Positive and negative financial flows are represented as conservative redistributions of funded sats:

$$(a, b, r) \to (a + d, b - d, r), \qquad (a, b, r) \to (a - d, b + d, r), \qquad (a, b, r) \to (a, b, r).$$

Each transition is defined only when the paying side has sufficient collateral.

Claim 10.1 (Financial Turing Completeness, Bounded Form)**.** *For every computable, collateral-bounded financial contract over oracle streams, there exists a cDLC family with Lazy Compression that implements the contract's terminal payoff on every terminating path for which the required preparation windows and oracle attestations occur.*

Proof. Let F be a computable, collateral-bounded financial contract. Since F is computable, there is a finite program computing its transition function δ and terminal payout function σ. Encode the program counter, working memory, and funded balances as the state z above.

For every compressed state class $\kappa(z)$ and oracle outcome x, prepare a cDLC branch whose bridge transaction maps z to $\delta(z, x)$ and whose adaptor point is S_x. By the cDLC activation equation, publication of s_x completes exactly the matching prepared branch, while wrong outcomes do not activate it. By construction, the child funding output carries the balance distribution of $\delta(z, x)$. □

11. Applications

A cDLC can express rolling contracts, re-hedging, periodic synthetic exposure, and conditional refinancing. These products are more precisely described as **renewable fixed-term contracts with bilateral extension rights**: either party may decline to extend the window, causing the fallback to execute.

12. Lightning Network Extension

This section shows that when the parties have already prepared a child state, the parent outcome scalar can also serve as a channel witness. The child state must still be negotiated in advance; Lightning only carries the conditional value transfer.

12.1 Channel Condition Model

A Lightning channel is modelled as $Q = (A, B, P)$ where A and B are balances and P is the set of pending conditional transfers. For a pending transfer $L = (\text{amount}, \text{lock}, \text{expiry})$, a redemption predicate $\text{redeem}(\text{lock}, w)$ determines settlement. The channel conserves capacity: $A + B = A' + B'$ in both cases.

12.2 HTLC Encoding

For each oracle outcome x, define $w_x = \text{enc}(s_x)$ and $h_x = H_{\text{pay}}(w_x)$. The HTLC lock is h_x; the oracle must precommit to h_x in its event announcement.

Claim 12.1 (HTLC Settlement)**.** *If the oracle publishes s_x and $h_x = H_{\text{pay}}(\text{enc}(s_x))$, the HTLC is redeemable by witness* $\text{enc}(s_x)$*.*

Proof. $\text{redeem}(h_x, w) = \text{true}$ iff $H_{\text{pay}}(w) = h_x$. Substituting $w = \text{enc}(s_x)$ holds by construction. □

Claim 12.2 (HTLC Outcome Isolation)**.** *If $H_{\text{pay}}(\text{enc}(s_y)) \neq H_{\text{pay}}(\text{enc}(s_x))$, then* $\text{enc}(s_y)$ *does not redeem the HTLC locked to h_x.*

Proof. Acceptance requires $H_{\text{pay}}(\text{enc}(s_y)) = h_x$, contradicting the premise under collision and second-preimage resistance. □

Claim 12.3 (HTLC–Adaptor Synchronisation)**.** *Publication of s_x both redeems the HTLC locked to h_x and completes the adaptor with point S_x.*

Proof. HTLC redemption follows from Claim 4.1. Adaptor completion: $(\hat{s} + s_x)G = (R^* - S_x + eP) + S_x = R^* + eP$. □

12.3 Routed HTLC cDLCs

For a route $N_0 \to \cdots \to N_k$ with hash lock h_x at every hop, the standard Lightning expiry ordering applies, with the additional constraint $\text{expiry}_k > \tau_{\text{attest}} + \delta$ for oracle observation margin δ.

Claim 12.4 (Routed HTLC Atomicity)**.** *A revealed* $\text{enc}(s_x)$ *propagates settlement back through the route; without it, the route times out.*

Proof. For every hop, $\text{redeem}(h_x, \text{enc}(s_x)) = \text{true}$ by Claim 4.1. Each node that learns the witness from its outgoing channel has the exact witness for its incoming channel. Without the witness, no hop can satisfy $H_{\text{pay}}(w) = h_x$ except by preimage or collision. □

12.4 Point-Locked Encoding

A point-locked transfer uses lock S_x and witness s_x with predicate $\text{redeem}_{\text{PTLC}}(S_x, s) = \text{true}$ iff $sG = S_x$. Since the ordinary DLC announcement already provides $S_x = s_x G$, no additional oracle hash commitment is required. PTLC deployment requires channel protocol changes not present in current major Lightning implementations.

Claim 12.5 (Point-Lock Settlement)**.** *Oracle publication of s_x redeems a transfer locked to S_x, since $s_x G = S_x$.*

Claim 12.6 (Point-Lock Outcome Isolation)**.** *If $s_yG \neq S_x$, then s_y does not redeem the point lock. In the prime-order group, $s_yG = s_xG$ implies $s_y = s_x \pmod n$.*

12.5 Routed Point-Locked cDLCs

Let the receiver's point be $T_k = S_x$. For each hop i, the node chooses a private tweak d_i and defines the upstream point $T_{i-1} = T_i + d_iG$. If downstream settlement reveals t_i with $t_iG = T_i$, the upstream witness is $t_{i-1} = t_i + d_i \pmod n$.

Claim 12.7 (Route Tweak Correctness)**.** $t_{i-1}G = (t_i + d_i)G = T_i + d_iG = T_{i-1}$.

12.6 Machine-Checked Lightning Model

The Lightning extension is modelled in Ada/SPARK, proof target `spark/lightning_cdlc_proofs.gpr`, using `type mod 97` for finite scalars and the ideal injective digest Hash_Of(s) = s. The target proves 13 companion claims covering HTLC and PTLC settlement, outcome isolation, route atomicity, tweak correctness, state activation and non-activation, channel balance conservation, and HTLC–adaptor synchronisation. The run analysed 51 SPARK subprograms and discharged 118 obligations with no unproved checks and no `pragma Assume` statements.

13. Limitations

The construction is an activation primitive, not a complete production protocol.

13.1 Cryptographic Assumptions

The construction assumes discrete logarithm hardness in secp256k1, Schnorr unforgeability, collision-resistant hash functions, and unique oracle nonces. Oracle nonce reuse in a cDLC graph compromises the entire downstream graph.

13.2 Oracle and Liveness Risk

If the oracle never publishes the outcome scalar, the child graph is not activated and funds follow the fallback policy. In HTLC mode, a wrong precommitted h_x causes liveness failure for the channel edge even when the DLC algebra is satisfied.

13.3 State and Negotiation Cost

Lazy preparation bounds maximum live retained state to LazyState(K), independent of total product depth D. Cumulative negotiation work scales with $D \cdot b$: long-running contracts require bilateral window extension at every period. If one party will not negotiate, the fallback executes.

Lazy Compression does not remove the need to prepare the live window before use. The financial-completeness theorem assumes that the required window has been materialized and that the oracle publishes the relevant attestations before the applicable deadlines.

13.4 Bitcoin Policy, Fees, and Timelocks

The proofs guarantee that the right scalar completes the right signature, not that a transaction confirms. Production systems must handle feerates, mempool policy, package relay, transaction pinning, anchor outputs, CPFP/RBF, dust limits, and timeout margins.

13.5 Privacy Limits

On-chain cDLCs are discreet at the script level under Taproot. In HTLC Lightning cDLCs, all hops observe the same h_x. PTLC cDLC channels resolve this via per-hop point tweaks.

13.6 Collateral Requirements

Every DLC node requires fully collateralized funding with no netting across positions. Capital locked in the funding output cannot be deployed elsewhere for the duration of the period.

13.7 Scope of the Formal Claim

Under the stated cryptographic assumptions, if all parties have exchanged and verified adaptors for every live edge in the current preparation window, at least one party retains the full adaptor state, and the oracle publishes exactly one valid outcome scalar before the bridge activation deadline, then that scalar activates the corresponding prepared child edge and cannot activate non-corresponding edges except through the failure modes stated above.

14. Conclusion

A DLC already contains a hidden scalar that becomes public only when a specific real-world outcome is attested. A cDLC uses that scalar twice: first to settle the parent contract, and second to complete adaptor signatures that fund the next contract.

The core equation is:

$$S_x = R_o + H(R_o \,\|\, V \,\|\, x)\, V, \qquad s_x G = S_x, \qquad s = \hat{s} + s_x \pmod{n}.$$

This is sufficient to build a graph of conditional Bitcoin transactions in which one DLC outcome activates another.

References

[1] T. Dryja, "Discreet Log Contracts," MIT Digital Currency Initiative, https://adiabat.github.io/dlc.pdf.

[2] BIP340, "Schnorr Signatures for secp256k1," https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki.

[3] "Discreet Log Contract Interoperability Specifications," https://github.com/discreetlogcontracts/dlcspecs.

[4] Bitcoin Optech, "Adaptor signatures," https://bitcoinops.org/en/topics/adaptor-signatures/.

[5] Lightning BOLT #2, https://github.com/lightning/bolts/blob/master/02-peer-protocol.md.

[6] Lightning BOLT #3, https://github.com/lightning/bolts/blob/master/03-transactions.md.

[7] Lightning BOLT #4, https://github.com/lightning/bolts/blob/master/04-onion-routing.md.

[8] Lightning BOLT #11, https://github.com/lightning/bolts/blob/master/11-payment-encoding.md.

[9] Bitcoin Optech, "Point Time Locked Contracts," https://bitcoinops.org/en/topics/ptlc/.